

Domain 1: Agentic Architecture & Orchestration — Self-Questioning List

CCAR-F Prep — Domain 1 (27% of scored content).

A. Foundations of Agentic Architecture

What is Agentic Architecture?

1. What makes an interaction "agentic" rather than a single request-response exchange?
2. What does it mean for Claude to "reason over intermediate results," and why is that the defining trait of agentic architecture?
3. In the Agent SDK's five-stage loop (receive prompt, evaluate, execute tools, repeat, return result), what happens if any one stage is skipped?

Core Components of an Agentic System

4. What distinguishes a "simple" user request from one that implicitly contains multiple concerns needing decomposition?
5. Why should a task objective be "clear enough for Claude to reason about success" — what happens architecturally when it isn't?
6. What is the difference between what a system prompt *tells* Claude to do and what it *guarantees* Claude will do?
7. Why should high-risk operations (e.g., blocking refunds above a threshold) never rely on the system prompt alone?
8. Why doesn't Claude "simply answer immediately" in an agentic system — what does it do instead when it lacks information?
9. What are the categories of built-in tools available to Claude Code and the Agent SDK (file, search, execution, web, discovery, orchestration)?
10. Why can poorly described tools lead to incorrect tool selection, duplicate work, or unsafe actions?
11. What is the separation of responsibility between Claude (deciding which tool to call) and the tool execution layer (deciding whether the call is allowed)?
12. Why should tool results be "concise, structured, and relevant" rather than raw and verbose?
13. What could go wrong if a tool result includes dozens of irrelevant fields instead of a structured summary?
14. Why does conversation history matter for continuity — what does Claude need to "remember" across turns that a single-turn system wouldn't?
15. What happens to a long-running agent's context window when it reads large files or runs verbose commands repeatedly?
16. What is the difference between the control loop and the stop condition — which one decides *whether* to continue, and which one decides *when* to end?
17. Why does the exam guide warn against parsing natural-language completion signals or checking assistant text as a stop indicator?
18. What is the difference between session state, session resumption, and session forking?

19. Why shouldn't every tool failure be treated the same way — what distinguishes a retryable error from one that requires escalation?
20. Why does a structured handoff summary matter when escalating to a human, rather than just forwarding the raw conversation?

Agentic System vs Traditional Workflow

21. What is the fundamental difference in decision-making between an agentic system and a traditional workflow?
22. What signals in a scenario indicate that agentic architecture is appropriate versus that a traditional, deterministic workflow is a better fit?
23. Why is a traditional workflow "best when the process is stable, deterministic, and easy to define in advance" — what happens if agentic architecture is used anyway?
24. Why is an agentic workflow better suited to investigation, context-sensitive judgment, or adaptive next steps — what would a traditional workflow fail to handle in these cases?

When Not to Use Agentic Architecture

25. What kinds of tasks are "better handled by fixed application logic" instead of an agent?
26. Why might traditional application logic be "safer, cheaper, easier to test, and easier to audit" for certain tasks?

Key Design Considerations

27. Why shouldn't an agent have every possible tool available to it by default?
28. What does "observability" mean for a production agent, and what specifically should the system record?
29. Under what conditions should an agent escalate rather than continue autonomously?
30. Why must long-running agents manage context carefully — what tools or patterns help with this (concise results, structured facts, subagents)?

Common Anti-Patterns

31. What goes wrong when every Claude application is treated as an agentic system?
32. Why is relying only on prompts for mandatory compliance rules considered an anti-pattern?
33. What risk is introduced by allowing state-changing tools to run without approval or validation?
34. Why is "assuming subagents automatically inherit all parent context" listed as an anti-pattern?
35. What happens to a system that lets an agent continue indefinitely without limits or budget controls?

B. Agentic Loop Lifecycle

Agent Loop Definition

36. What are the five stages of the agentic loop as described in the Agent SDK documentation?
37. Why is an agentic loop described as "iterative" rather than a one-shot request-response system?
38. What defines a "turn" in Claude's agentic loop, and when does the loop actually end?

39. Why is the agentic loop treated as an "architectural pattern" rather than just a programming convenience?

Stop Reasons as Control Signals

40. What is the difference between `stop_reason: "tool_use"` and `stop_reason: "end_turn"` — what should the application do in each case?
41. Why is `stop_reason` described as part of every *successful* response, distinct from an API failure?
42. What other stop reasons exist besides `tool_use` and `end_turn` (e.g., `max_tokens`, `pause_turn`, `model_context_window_exceeded`), and how should each be handled?
43. Why can Claude return an empty response with `stop_reason: "end_turn"` even when the task isn't actually complete — what implementation mistake causes this?

Returning Tool Results and Updating Conversation History

44. Why must tool results be appended to conversation history for the loop to function correctly?
45. What is the difference between a client tool and a server tool, and why does only the client-tool path require the application to manually continue the conversation?
46. What two formatting requirements does Anthropic specify for tool results in the message history (position relative to the tool-use message, and order within the content array)?
47. What happens if tool result ordering rules are violated?
48. Why does the exam guide emphasize "conversation history updates" rather than just "tool execution" as the real unit of the agentic loop?
49. Why can adding free text immediately after a `tool_result` lead to an empty `"end_turn"` response?

Model-Driven Orchestration Versus Preconfigured Flows

50. What is the difference between model-driven tool selection and a pre-configured, hard-coded tool sequence?
51. Under the default `tool_choice: {"type": "auto"}` setting, what determines whether Claude calls a tool versus responds directly?
52. Why does a hard-coded flow risk calling the same tools in the same order "regardless of what the intermediate results show"?
53. Does "model-driven" mean "uncontrolled" — what constraints can still apply to a model-driven loop?

Anti-Patterns and Implementation Guidance

54. Why is checking assistant text content for phrases like "I'm done" considered an anti-pattern for loop termination?
55. Why shouldn't `max_turns` be treated as the primary definition of task completion?
56. What does `max_turns` actually count, and what is its correct role (safety rail vs. completion detector)?
57. Why are message-ordering mistakes with tool results considered a "class of anti-pattern" rather than a cosmetic issue?
58. What is the exam-ready rule of thumb for using `stop_reason`, `max_turns`, and message ordering together?

C. Coordinator-Subagent Orchestration

Coordinator as Hub and Subagents as Spokes

- 59. Why is the recommended architecture described as "hub-and-spoke," with the coordinator at the center?
- 60. What three benefits does Anthropic's documentation attribute to subagents (context isolation, parallelization, specialized instructions)?
- 61. Why can't subagents currently spawn their own subagents in the standard SDK model, and what does that imply for who remains the single orchestrator?
- 62. Why is a hub-and-spoke design preferred over a peer-to-peer multi-agent mesh — what does a mesh obscure or complicate?

Context Isolation and Explicit Handoff Design

- 63. Why don't subagents automatically inherit the coordinator's conversation history?
- 64. What is the only direct channel from parent to subagent, and what must the coordinator include in it?
- 65. What happens if the coordinator delegates with something like "analyze the issue from earlier" without restating the relevant facts?
- 66. What does the parent coordinator *not* receive from a subagent by default — and why does this matter for provenance or citations?
- 67. Why should a subagent's output be treated as a "formal handoff artifact" rather than a casual chat reply when it will later be merged with other findings?

Dynamic Delegation, Partitioning, and Synthesis

- 68. Why should a coordinator dynamically select which subagents to invoke rather than always routing every request through a fixed pipeline?
- 69. What determines whether Claude invokes a particular subagent — and how can a developer force a specific one?
- 70. What is the risk of overly narrow decomposition by a coordinator (as illustrated by the creative-industries example)?
- 71. What does the coordinator do differently when a coverage review reveals a thin or missing dimension — does it rerun the entire workflow or redelegate a targeted follow-up?
- 72. Why is routing all communication through the coordinator "the cleanest way to maintain observability"?
- 73. At what scale does the conversational coordinator-subagent pattern start to strain, and what pattern is recommended instead?

Common Pitfalls and Authoritative Sources

- 74. Why is treating decomposition as "a purely mechanical split" rather than a coverage design problem the most common failure mode?
- 75. Why is assuming subagents share the parent's state implicitly listed as a common mistake?
- 76. Why is letting every request traverse the entire pipeline, regardless of complexity, a pitfall?
- 77. Why does over-permissive tool design in subagents create a governance issue, not just a technical one?

78. Why can permissive modes like `bypassPermissions` not be relaxed per subagent once inherited?

D. Subagent Invocation & Context Passing

How Subagents Are Spawned

- 79. Through which tool are subagents invoked in the current Claude Agent SDK?
- 80. What must be included in `allowedTools` for a parent agent to auto-approve subagent invocation?
- 81. What is the historical naming relationship between the `Task` tool and the `Agent` tool, and why does it matter when interpreting older exam phrasing?
- 82. What happens if no custom subagent is defined — can Claude still invoke a subagent?
- 83. What is the difference between letting Claude choose a subagent dynamically (based on description) versus explicitly naming one in the prompt?

What Context a Subagent Does and Does Not Receive

- 84. What is the single most important fact about what a subagent receives from its parent?
- 85. What specifically must the parent include directly in the delegation prompt for the subagent to act correctly?
- 86. Besides the delegation message, what else does a subagent typically receive (system prompt, tool definitions, and in Claude Code, possibly `CLAUDE.md`, memory, session info)?
- 87. Which built-in subagents skip `CLAUDE.md` and git status entirely, and what does that mean for a project rule that must reach them?
- 88. Why is "continue the earlier analysis" a weak delegation prompt?
- 89. What does the parent coordinator receive back from a subagent by default — the full transcript, or only the final message?
- 90. What must a coordinator explicitly request if it needs the subagent's exact wording preserved rather than summarized?

How to Define Subagents Well

- 91. What are the two minimum required fields in an `AgentDefinition`?
- 92. What is the functional difference between the `description` field and the `prompt` field of a subagent?
- 93. What happens to a subagent's tool access if the `tools` field is omitted entirely?
- 94. What happens if both `tools` and `disallowedTools` are specified — which is applied first?
- 95. Why does a reviewer subagent that only needs Read, Grep, and Glob pose a risk if tool restriction isn't configured deliberately?
- 96. Why is a vague description like "research agent" weaker than a specific one like "finds and summarizes policy evidence with citation metadata"?
- 97. What is the discrepancy between the current SDK docs ("subagents cannot spawn their own subagents") and a more recent Claude Code changelog entry — and how should that tension be handled for exam purposes versus live systems?

Forks, Resumes, and Parallel Work

- 98. What is the difference between a session-level fork and a subagent-level fork?

99. When creating a new session as a fork, what does it start with, and what happens to the original session?
 100. Why would a forked subagent be useful when a named subagent "would need too much background to be effective"?
 101. What does a forked subagent see (system prompt, tools, model, message history) compared to a freshly spawned subagent?
 102. What does **isolation: "worktree"** change about where a fork's file edits happen?
 103. What is the difference between a forked subagent and a resumed subagent in terms of what history each retains?
 104. What is the documented pattern for resuming a subagent (capturing **session_id**, extracting **agentId**, resuming with a prompt)?
 105. Why can't the built-in Explore and Plan agents be resumed, and what does that imply for choosing a subagent type when resumability matters?
 106. Why does running multiple subagents concurrently reduce total wall-clock time compared to running them one after another?
-

E. Multi-Step Workflow Enforcement & Handoff

Programmatic Enforcement vs. Prompt Guidance

107. What is the difference between what prompt guidance can influence and what programmatic enforcement can guarantee?
108. For which categories of operations is prompt guidance alone considered insufficient (financial, legal, security-related, customer-impacting)?
109. Given a table of requirements (tone, clarifying questions, identity verification, blocking a tool call, escalation, handoff), which control type does each one need?
110. What is the common exam trap involving "improving the prompt" when a workflow actually requires deterministic compliance?

Deterministic Compliance

111. Why doesn't a prompt instruction like "always call `get_customer` before `process_refund`" guarantee compliance on its own?
112. What does a prerequisite gate add on top of a prompt instruction to make the workflow reliable?
113. What does the system return when a tool call is blocked by a gate — what fields does a well-designed block response include (**allowed**, **reason**, **next_action**)?

Prerequisite Gate Pattern

114. In a refund workflow with four sequential steps, what must be verified before **process_refund** is allowed to execute?
115. Can Claude still reason about the customer's issue and choose tools freely within a prerequisite-gated workflow — what exactly is restricted?
116. What is the difference in gate logic between checks that pass (allowing the tool call) and checks that fail (blocking and returning next steps)?

Hooks for Enforcement

- 117. When are hooks preferred over prerequisite gates — what do hooks add that gates alone do not (inspecting or modifying a call before execution)?
 - 118. What kinds of business rules are commonly enforced with hooks (blocking large refunds, restricted field changes, human approval, destructive actions, input validation)?
-

F. Agent SDK Hooks for Tool Interception & Data Normalization

What Hooks Are

- 119. How does Anthropic define a hook, and why are hooks described as "not suggestions to the model"?
- 120. Why are hooks the correct enforcement mechanism whenever a rule must apply on every tool call, not just most of them?

PreToolUse Hooks

- 121. At what point in the tool execution lifecycle does a **PreToolUse** hook run?
- 122. What three actions can a **PreToolUse** hook take on a tool call (allow, block, modify)?
- 123. What happens when a **PreToolUse** hook returns exit code 2 — does **bypassPermissions** mode override it?
- 124. Why does this make **PreToolUse** hooks "the strongest enforcement mechanism available in Claude Code"?

PostToolUse Hooks

- 125. At what point does a **PostToolUse** hook run, and what information does it receive (tool name, input, output)?
- 126. What kinds of tasks is a **PostToolUse** hook suited for (linting after edits, schema validation, audit logging)?
- 127. In the ESLint example, what determines which tool calls trigger the hook, and does Claude's own reasoning affect whether the hook fires?

Hook Matchers

- 128. What does a hook matcher control?
- 129. What is the difference between a matcher targeting a single tool, a pipe-separated list of tools, and an empty matcher?
- 130. What goes wrong with an overly broad matcher versus an overly narrow one?
- 131. How should a matcher be scoped if a linting rule must apply to every file-writing tool?

Using Hooks for Data Normalization

- 132. What is "data normalization" in the context of **PostToolUse** hooks?
- 133. What two benefits does normalization provide (reduced cognitive load on the model, consistent downstream formatting)?
- 134. What kinds of transformations does a normalization hook typically perform (stripping fields, converting timestamps, normalizing status codes, extracting relevant content)?

Hooks vs. CLAUDE.md — The Enforcement Spectrum

135. In a scenario where a CLAUDE.md rule is followed "most of the time but not always," what is the correct diagnostic?
 136. Why can't strengthening the wording of a CLAUDE.md instruction (adding "IMPORTANT" or "ALWAYS") eliminate violations entirely?
 137. Where do hooks sit on the enforcement spectrum relative to CLAUDE.md, and why can't they be bypassed by prompt context or model reasoning?
 138. Where do path-scoped rules and skills fall on this spectrum — are they closer to CLAUDE.md or to hooks?
 139. What pattern in a scenario signals that a hook (not a stronger CLAUDE.md instruction) is the needed fix?
-

G. Task Decomposition Strategies

What Task Decomposition Is

140. Why does the quality of decomposition directly determine the quality of the final synthesized output?
141. What three things does the exam expect a candidate to evaluate in a given decomposition (coverage, independence of partitions, verifiability)?

Decomposition Dimensions

142. What is domain decomposition, and what makes a domain-decomposition partition "vague" versus genuinely aligned with the subject matter?
143. In the creative-industries example, what specifically goes wrong if the coordinator decomposes into only graphic design, digital art, and photography?
144. What is source-type decomposition, and what problem does it avoid (context window saturation from mixing raw gathering with synthesis)?
145. What is functional decomposition, and for what kind of workflow is it most appropriate (sequential pipelines with stage-dependent outputs)?

Parallelism vs. Sequentiality

146. What condition makes a decomposition suitable for parallel execution (subtask independence)?
147. What condition requires sequential decomposition instead (one stage's output required as input to the next)?
148. What happens if sequential stages are forced to run in parallel?
149. What does a coordinator need to manage in a mixed workflow that combines both parallel and sequential elements?

Verifying Decomposition Completeness

150. What are the two properties of a correctly decomposed task (distinctness and collective sufficiency)?
151. What does a coverage gap in decomposition cause in the final synthesis?

- 152. What is the most practical way for a coordinator to verify collective sufficiency before returning a result to the user?
- 153. If a requested dimension is missing or thinly supported, what are the coordinator's two options (re-delegate a targeted follow-up, or acknowledge the gap)?

When Not to Decompose

- 154. What coordination overhead does decomposition introduce (context handoff, parsing/integrating results, added latency)?
 - 155. What is the correct threshold for deciding to decompose a task (context isolation need or specialization need)?
 - 156. If a coordinator's final synthesis comes back incomplete or imbalanced, what is almost always the root cause — and what is the correct fix?
-

H. Session State, Resumption & Forking

Why Session State Matters

- 157. Why is session state a non-issue in simple single-turn interactions but an architectural concern in long-running agentic workflows?
- 158. What must a long-running agent be able to do that a single-turn interaction never needs to (maintain continuity, recover from interruption, explore alternatives)?

Session Resumption

- 159. How is a prior session resumed, and what does resumption preserve that a summarized re-injection of context does not?
- 160. What is the difference between resuming a session and starting a new session with context manually re-injected?
- 161. When is session resumption the correct pattern versus starting fresh?

Session Forking

- 162. When is forking the appropriate choice (evaluating multiple approaches, uncertain next steps, testing a risky operation in isolation)?
- 163. What does a fork preserve up to the fork point, and what accumulates independently afterward in each branch?
- 164. In the two-refactoring-approaches scenario, why is "session forking" the correct answer instead of creating two separate sessions from scratch?

Scratchpad Files for Long Sessions

- 165. What happens to earlier information in a long-running session once the context window fills up?
- 166. What is the scratchpad file pattern, and what does it allow an agent to do that raw context accumulation does not?
- 167. What is the relationship between `/clear` and a scratchpad file — what problem does each solve on its own, and what problem do they solve together?
- 168. Why doesn't session resumption alone address context window exhaustion *within* a single session?

Sub-Agent Spawning for Context Management

- 169. How does spawning a subagent for a context-intensive sub-task differ from using a scratchpad file to manage the same problem?
- 170. When is subagent spawning preferable to a scratchpad (many large files, verbose commands, multi-step searches)?
- 171. What stays isolated inside the subagent's own session, and what is the only thing that returns to the coordinator?

The /clear Command

- 172. What does `/clear` actually reset, and what does it leave untouched (the session itself)?
- 173. What happens to the agent's memory of prior tool calls after `/clear` if no scratchpad file was used?
- 174. Given a long-running task that is "degrading in quality or becoming incoherent," what is the root cause usually diagnosed as, and what is the correct response depending on whether the task can be broken into bounded sub-tasks?

Cross-Cutting Self-Questioning Prompts for Domain 1

- 175. For any scenario describing "intermittent" rule violations despite clear instructions — is this signaling an advisory-vs-deterministic gap, and if so, is a hook or a prerequisite gate the correct fix?
- 176. For any scenario describing an agent that "doesn't know when to stop" — is the fix a better stop-reason check, or is the scenario actually testing `max_turns` misuse as a completion signal?
- 177. For any scenario describing a coordinator whose final output is incomplete or imbalanced — is the root cause decomposition design rather than insufficient instruction?
- 178. For any scenario describing a subagent that "hallucinates" or acts on wrong assumptions — did the coordinator's delegation prompt fail to pass required context explicitly?
- 179. For any scenario describing a long session that "degrades" — is the fix `/clear` + scratchpad, or subagent spawning, and what in the scenario determines which one applies?
- 180. For any scenario comparing two candidate solutions where one relies on prompt wording and the other on a system-level mechanism (hook, gate, permission) — which one actually guarantees the required behavior?